

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUADA - PEC 2

Presentación

Esta segunda PEC profundiza en las arquitecturas de software e introduce el desarrollo de software basado en componentes. La PEC 2 consiste en refinar la especificación de la PEC 1 mediante su descomposición en componentes de software. La actividad alcanza los contenidos estudiados en los módulos 2, 3 y 4 de la asignatura, y repasa conceptos aprendidos en el módulo 1.

Competencias

En esta PEC se trabaja la siguiente competencia del Grado de Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

Los objetivos concretos de esta PEC son:

- Entender los diferentes puntos de vista a considerar en el desarrollo de este tipo de aplicaciones.
- Conocer los diferentes estilos arquitectónicos existentes y saber definir la arquitectura de software más adecuada según las características particulares de cada aplicación.
- Conocer la programación orientada a componentes como técnica de implementación de las arquitecturas de software.

Descripción de la PEC a realizar

La administración pública que quiere poner en marcha el proyecto **MyHealth** ha evaluado vuestra propuesta y ha quedado muy satisfecho con la especificación del punto de vista de la información y con la primera evaluación arquitectónica. Por este motivo ha decidido que llevéis a cabo el diseño y, posiblemente, la posterior implementación.

Hay varias decisiones inherentes al estilo arquitectónico que habéis propuesto o bien que ya formaban parte de los requisitos iniciales y que el cliente ha aceptado:

- Los clientes tienen que poder acceder a la aplicación sin tener que instalar ningún software especial, únicamente un navegador o la aplicación móvil correspondiente.
- La aplicación tendrá una arquitectura en capas.
- Tendremos tres capas: capa de presentación, capa de negocio y capa de integración (también denominada de administración de datos).
- Se propone implementar las tres capas usando una tecnología de componentes distribuida.

Después de una segunda ronda de reuniones con algunos de los *stakeholders* involucrados, se han definido con fuerza más precisión los requisitos funcionales y ya tenemos claras las funcionalidades que, en una primera fase, tiene que ofrecer **MyHealth**.

Para cada una de las funcionalidades se especifican los usuarios que hacen uso de ellas: Paciente (P), Especialista (S), Médico de familia (F), Administrador (A) y Usuario no registrado.

- Identificarse en el sistema (login) (P, S, F, A).
- Salir del sistema (logout) (P, S, F, A)
- Registrarse en el sistema (Usuario no registrado)
- Modificar los datos personales (P, S, F)
- Cambiar de médico de familia (P)
- Cambiar de CAP - centro de atención primaria (F)
- Pedir hora para una visita (P)
- Modificar la fecha y hora de una visita (A, P)
- Anular una visita (P)
- Añadir el resultado de la visita (F)
- Añadir una prueba médica (S)
- Consultar una prueba médica (P, S, F)
- Hacer una pregunta al médico de familia (P)
- Responder a una pregunta de un paciente (F)
- Ver todas las preguntas pendientes de respuesta (F)
- Gestionar los tipos de especialidad (alta, consulta, baja, modificación) (A)
- Gestionar los centros de atención primaria (alta, consulta, baja, modificación) (A)
- Ver todas las visitas que tiene programadas en un determinado día (F)
- Ver todos los médicos de familia de un CAP con el número de pacientes (A)

La capa de presentación tiene que implementar las diferentes interfaces para ofrecer las funcionalidades descritas anteriormente.

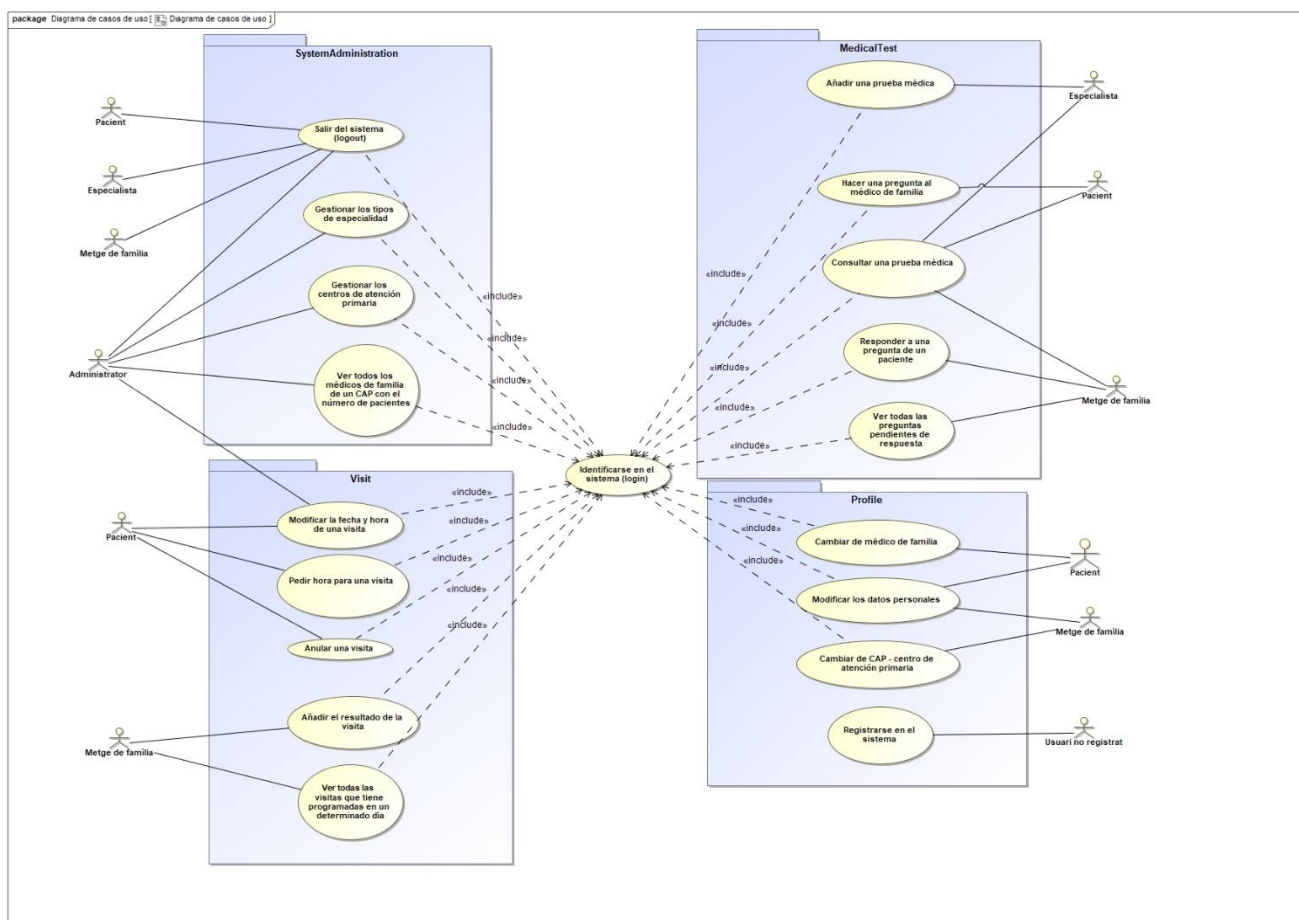
La capa de negocio tiene que implementar las funcionalidades descritas con una arquitectura suficientemente flexible porque en un futuro se puedan integrar nuevas funcionalidades y extender las actuales. En esta capa también se tiene que tener en cuenta los componentes que se encargan de comunicarse con el sistema externo a **MyHealth** especializado al almacenar las imágenes en alta definición de determinadas pruebas médicas.

La capa de integración gestionará la información localmente persistente de la aplicación. Tendríais que gestionar toda la información necesaria para poder proporcionar las funcionalidades descritas anteriormente.

Ejercicio 1 (1,5 puntos)

Haced el diagrama de casos de uso de las funcionalidades requeridas agrupando los casos de uso en paquetes en base a la funcionalidad que describen. Tened cuidado con la granularidad de los casos de uso: puede ser que varias de las funcionalidades descritas se integren dentro de un mismo caso de uso o bien que alguna de las funcionalidades expuestas den lugar a varios casos de uso.

Solución:



Notas:

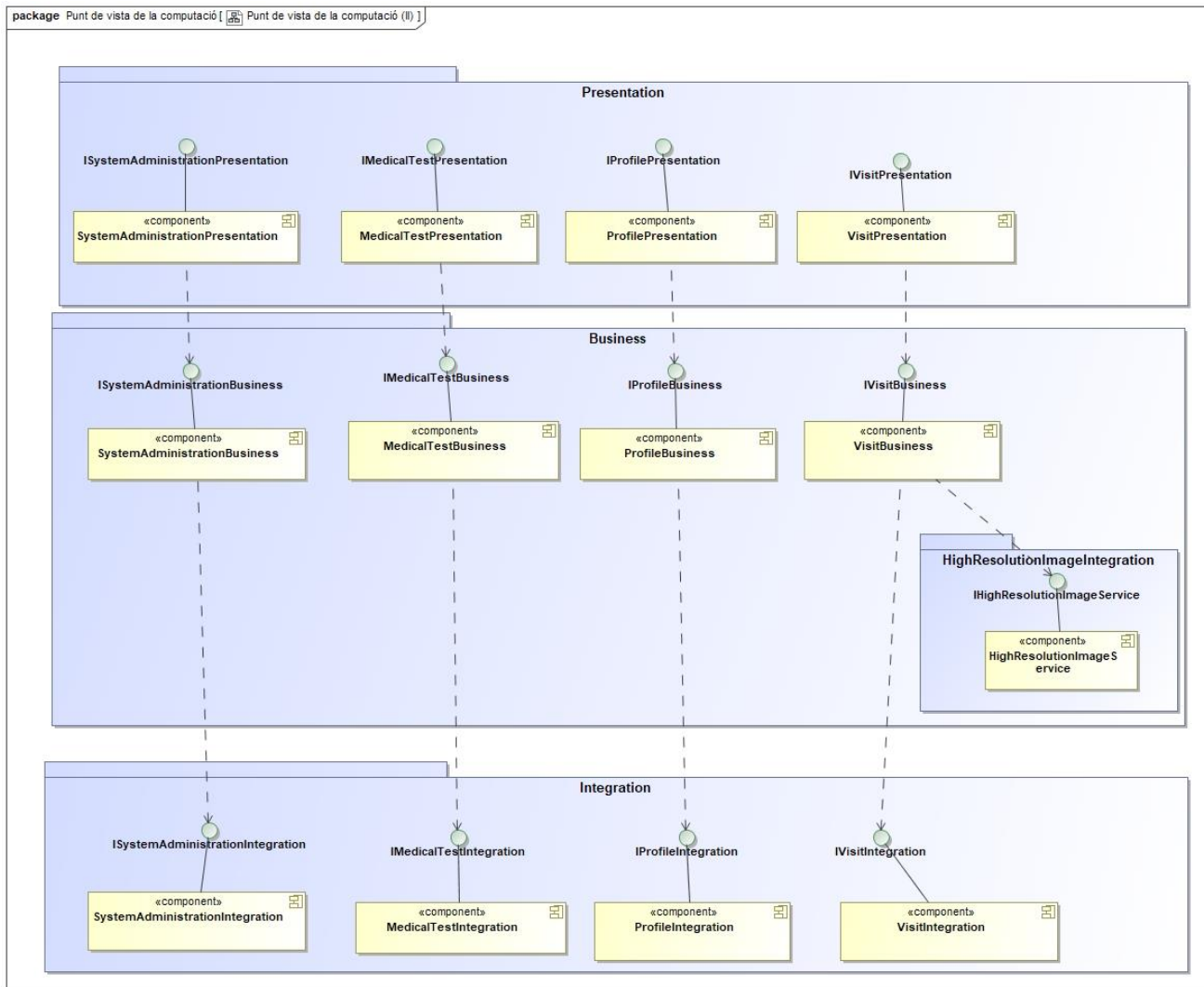
- La agrupación propuesta sólo es una de las infinitas maneras de agrupar las funcionalidades.
- Se han puesto los actores varias veces al diagrama por legibilidad.

Ejercicio 2 (3,5 puntos)

Representar el punto de vista de la computación, es decir, el diseño interno de la funcionalidad de la aplicación mediante diagramas de componentes dentro de una arquitectura en capas. Cada capa a nivel lógico se representa en UML como un paquete. Dentro de cada paquete, se identifican los componentes lógicos (componentes de grano grueso, no confundir con los componentes de software que los implementarán, estos forman parte del punto de vista de la ingeniería) con sus respectivas interfaces (tomar como referencia la figura 18 del material del módulo didáctico “Arquitectura del software”: “Ejemplo de representación de la arquitectura de tres capas para el ejemplo de la banca electrónica”).

Es necesario describir las interfaces de todos los componentes mediante interfaces UML donde se muestre la firma de los servicios que ofrece cada componente.

Solución:



Por motivos de legibilidad del diagrama se han mostrado las interfaces colapsadas, a continuación tenéis todas las definiciones:

Presentation:

package Punt de vista de la computació [ Presentation]

ISystemAdministrationPresentation

```
+login( id : String, pwd : String )
+logout()
+addCAP( name : String, location : Location )
+deleteCAP( name : String )
+listAllCAPs()
+updateCAP( name : String, location : Location )
+registerAdministrator( password : String, email : String )
+listAllFamilyDoctorsByCAP( cap : PrimaryHealthCareCenter )
+getMedicalSpecialty( name : String )
+deleteMedicalSpecialty( name : String )
+listAllMedicalSpecialties()
+updateMedicalSpecialty( name : String, description : String )
+addMedicalSpecialty( name : String, description : String )
+getCAP( name : String )
```

IMedicalTestPresentation

```
+askQuestion( id : ProfessionalNumber, title : String, message : String )
+answerQuestion( question : String, response : String )
+listAllPendingQuestions( id : ProfessionalNumber )
+addMedicalTest( id : int, date : Date, time : Time, type : TestType, observations : String )
+getMedicalTest( id : int )
+getQuestion( question : String )
```

IProfilePresentation

```
+changeFamilyDoctor( id : PersonalIdentificationCode, newDoctor : ProfessionalNumber )
+registerPacient( id : PersonalIdentificationCode, nif : String, name : String, surname : String, password : String, email : String )
+registerSpecialistDoctor( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, specialty : MedicalSpecialty )
+registerFamilyDoctor1( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, cap )
+updatePacientData( id : PersonalIdentificationCode, nif : String, name : String, surname : String, password : String, email : String )
+updateSpecialistDoctorData( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, specialty : MedicalSpecialty )
+updateFamilyDoctorData( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, cap )
+changePrimaryHealthCareCenter( id : ProfessionalNumber, newCenter : PrimaryHealthCareCenter )
```

IVisitPresentation

```
+addVisit( id : int, date : Date, time : Time, observations : String )
+updateVisit( id : int, date : Date, time : Time )
+removeVisit( id : int )
+addResultToVisit( id : int, result : String )
+listAllScheduledVisits( id : ProfessionalNumber, date : Date )
+getVisit( id : int )
```


Business:

package Punt de vista de la computació [ Business]

ISystemAdministrationBusiness

```
+login( id : String, pwd : String )
+logout()
+addCAP( name : String, location : Location )
+deleteCAP( name : String )
+listAllCAPs()
+updateCAP( name : String, location : Location )
+registerAdministrator( password : String, email : String )
+listAllFamilyDoctorsByCAP( cap : PrimaryHealthCareCenter )
+getMedicalSpecialty( name : String )
+deleteMedicalSpecialty( name : String )
+listAllMedicalSpecialties()
+updateMedicalSpecialty( name : String, description : String )
+addMedicalSpecialty( name : String, description : String )
+getCAP( name : String )
```

IMedicalTestBusiness

```
+askQuestion( id : ProfessionalNumber, title : String, message : String )
+answerQuestion( question : String, response : String )
+listAllPendingQuestions( id : ProfessionalNumber )
+addMedicalTest( id : int, date : Date, time : Time, type : TestType, observations : String )
+getMedicalTest( id : int )
+getQuestion( question : String )
```

IProfileBusiness

```
+changeFamilyDoctor( id : PersonalIdentificationCode, newDoctor : ProfessionalNumber )
+registerPacient( id : PersonalIdentificationCode, nif : String, name : String, surname : String, password : String, email : String )
+registerSpecialistDoctor( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, specialty : MedicalSpecialty )
+registerFamilyDoctor1( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, cap )
+updatePacientData( id : PersonalIdentificationCode, nif : String, name : String, surname : String, password : String, email : String )
+updateSpecialistDoctorData( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, specialty : MedicalSpecialty )
+updateFamilyDoctorData( id : ProfessionalNumber, nif : String, name : String, surname : String, password : String, email : String, cap )
+changePrimaryHealthCareCenter( id : ProfessionalNumber, newCenter : PrimaryHealthCareCenter )
```

IVisitBusiness

```
+addVisit( id : int, date : Date, time : Time, observations : String )
+updateVisit( id : int, date : Date, time : Time )
+removeVisit( id : int )
+addResultToVisit( id : int, result : String )
+listAllScheduledVisits( id : ProfessionalNumber, date : Date )
+getVisit( id : int )
```

Integration:



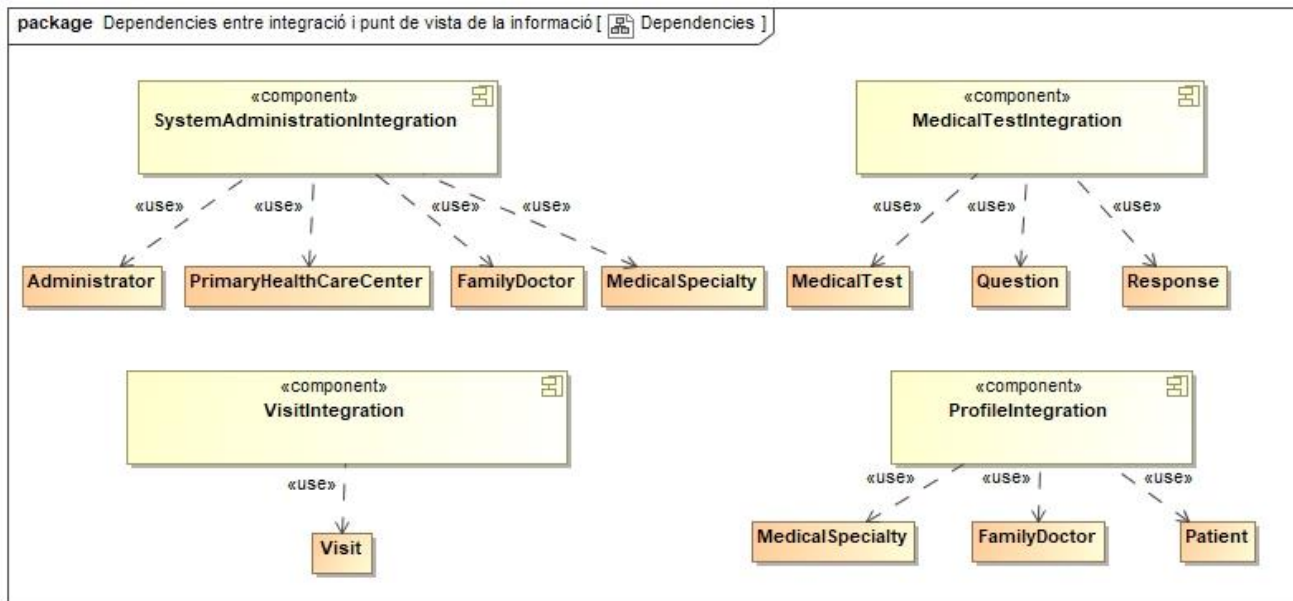
Notas:

- La agrupación propuesta sólo es una de las infinitas maneras de agrupar las funcionalidades en componentes, cualquier agrupación que hayáis elegido y tenga una cierta coherencia se considerará correcto.

Ejercicio 3 (2 puntos)

Modelar con un diagrama de componentes las dependencias de cada componente de la capa de integración con las clases del punto de vista de la información que el componente necesita.

Solución:



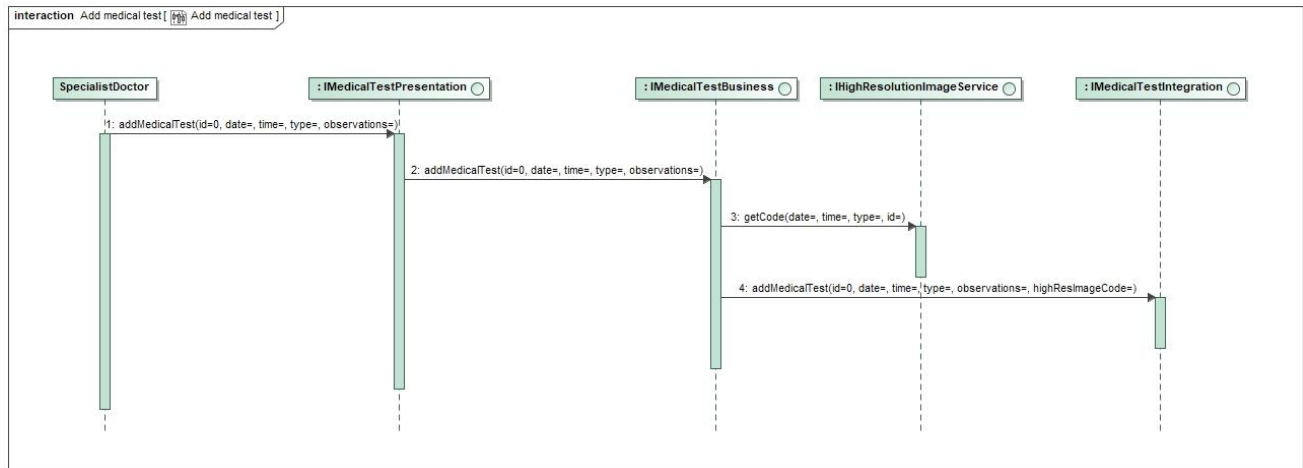
Notas:

- Las dependencias de cada componente de la capa de integración con las clases del punto de vista de la información propuesta dependen muy estrechamente de la división en componentes que hayáis hecho en el ejercicio anterior. Vuestra solución tiene que ser coherente con esta división.

Ejercicio 4 (2 puntos)

Trabajar el diseño externo con un diagrama de interacción que modele la interacción entre objetos dentro de una capa y entre capas a partir del momento en que un especialista pide ejecutar la siguiente funcionalidad: *Dar de alta una nueva prueba médica*. Considerad que la prueba médica tiene una imagen en otra resolución asociada.

Solución:



Ejercicio 5 (1 punto)

Explicáis con vuestras palabras la relación existente entre el software intermediario de aplicaciones distribuidas en el mundo Java, EJB y RMI (máximo media página).

Solución:

Ver los puntos 1.2.2, 1.3 y 1.3.2 del módulo 4.

Recursos

Recursos Básicos

- Módulo didáctico 1: Diseño de aplicaciones distribuidas
- Módulo didáctico 2: Arquitectura del software
- Módulo didáctico 3: Desarrollo de software basado en componentes
- Módulo didáctico 4: Introducción a las plataformas distribuidas

Criterios de evaluación previos

- La PEC se tiene que resolver de forma **estrictamente individual**. En caso de detectar copias se penalizará la actividad con una D como nota.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSDPEC2.pdf*.

Este documento se entregará en el espacio de Entrega y Registro de Evaluación Continuada (REC) del aula antes de las **23:59 horas del día 16 de abril de 2019**. No se aceptarán entregas fuera de plazo.